EECS 836: Machine Learning

Zijun Yao

Assistant Professor, EECS Department

The University of Kansas

Anonymous Feedback (active till Feb 3)

Active poll

Join at

slido.com

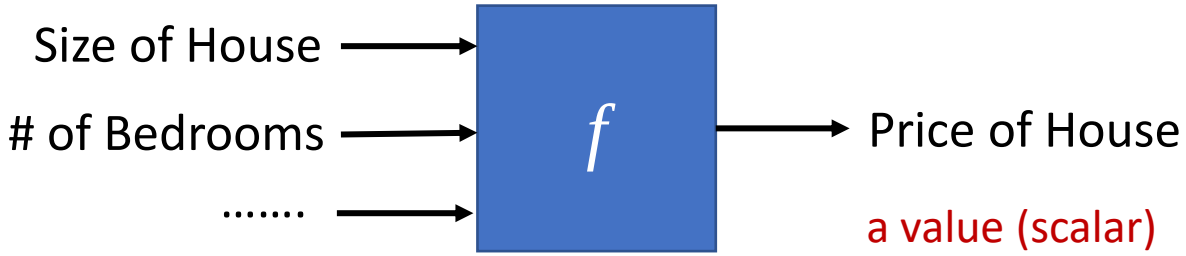
#8512 032

Midway Check-In: How is 836 Going for You?

Agenda

- Logistic Regression model
  - Model definition
  - Loss function
  - Optimizing parameters
- Model Evaluation
  - Metrics
  - Methods

House price prediction - regression

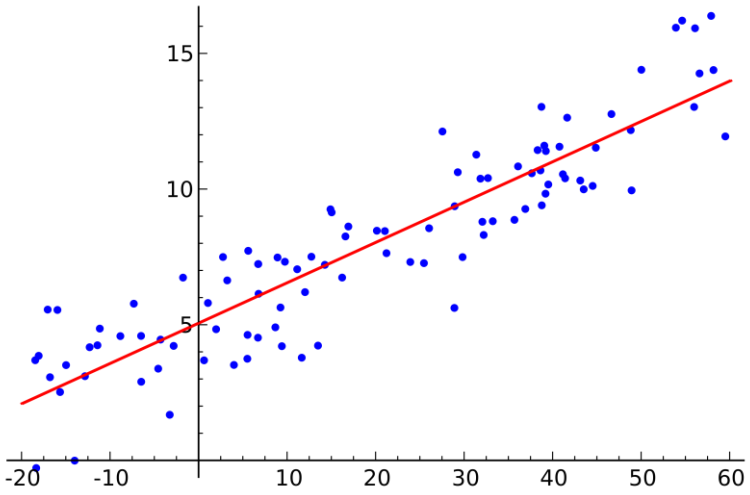


Size of House	Lot Size (acre)	# of Bedrooms	# of Bathrooms	Price of House
950	2.5	2	1	$127,325
1,535	1.5	2	2	$156,570
1,605	2.25	3	1.5	$158,895
1,905	2.5	2	1.5	$200,025
2,057	2.25	3	2	$230,384
2,227	2.75	3	2	$233,835
3,150	1	4	2	$261,420
3,620	3	4	3	$433,500

Feature x Target y

Linear regression recap

- The problem of predicting continuous values is called regression problem
- Given
  - Data $\boldsymbol{X} = \left\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(n)}\right\}$ where $\boldsymbol{x}^{(i)} \in \mathbb{R}^d$
  - Corresponding labels $\boldsymbol{y} = \left\{y^{(1)}, \ldots, y^{(n)}\right\}$ where $y^{(i)} \in \mathbb{R}$
- Find a continuous function that models the continuous points



Model definition

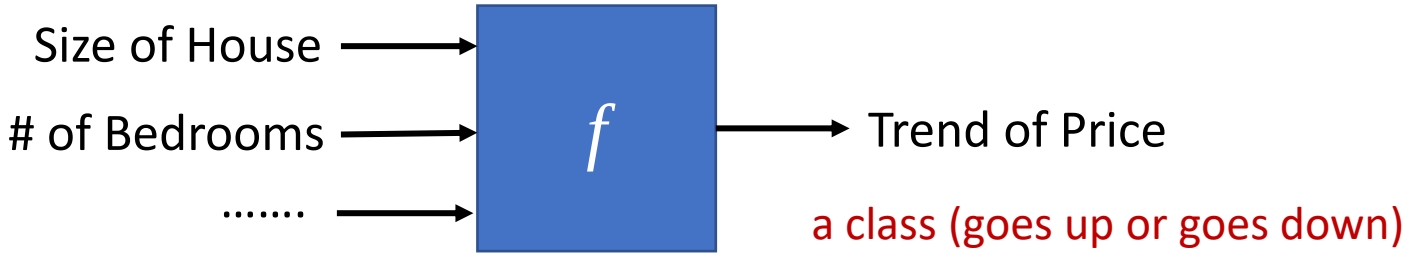
$$\hat{y} = w_0 x_0 + w_1 x_1 + w_2 x_2 + \ldots + w_d x_d$$

w_0 is bias b, x_0 is 1 for all data

Loss function

$$L(\mathbf{w}, b) = \sum_{i=1}^{n} (y^{(i)} - \hat{y}^{(i)})^2$$

House price prediction - classification



Size of House	Lot Size (acre)	# of Bedrooms	# of Bathrooms	Trend of Price
950	2.5	2	1	Down
1,535	1.5	2	2	Up
1,605	2.25	3	1.5	Down
1,905	2.5	2	1.5	Down
2,057	2.25	3	2	Up
2,227	2.75	3	2	Up
3,150	1	4	2	Down
3,620	3	4	3	Up

Feature x Target y

Linear classifiers - 3 steps

- Model definition:

$\boldsymbol{x}$ ⟶

$z(\boldsymbol{x}) > 0$	Output = class 1
$else$	Output = class 0

- Loss function: how good is a classifier?

$$L(f) = \sum_i \delta\left(f\left(x^{(i)}\right) \neq y^{(i)}\right)$$

The number of times *f*(x) get incorrect results on training data.

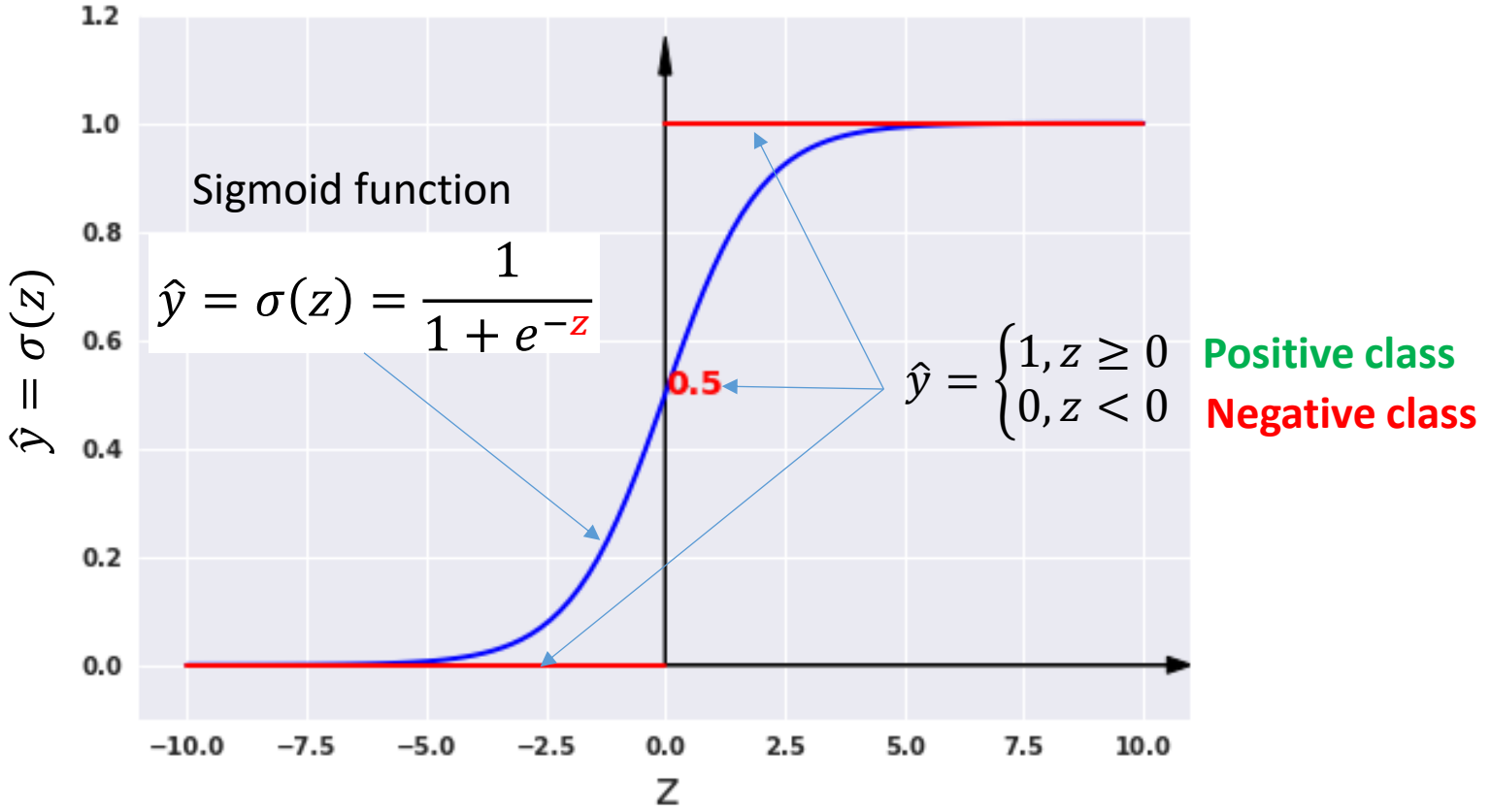
- Find the best classifier parameters: optimization algorithm

Step 1: Logistic regression definition

- Weight all features using linear regression

$$z = w_0 x_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d$$ (w_0 is bias b, x_0 is 1)

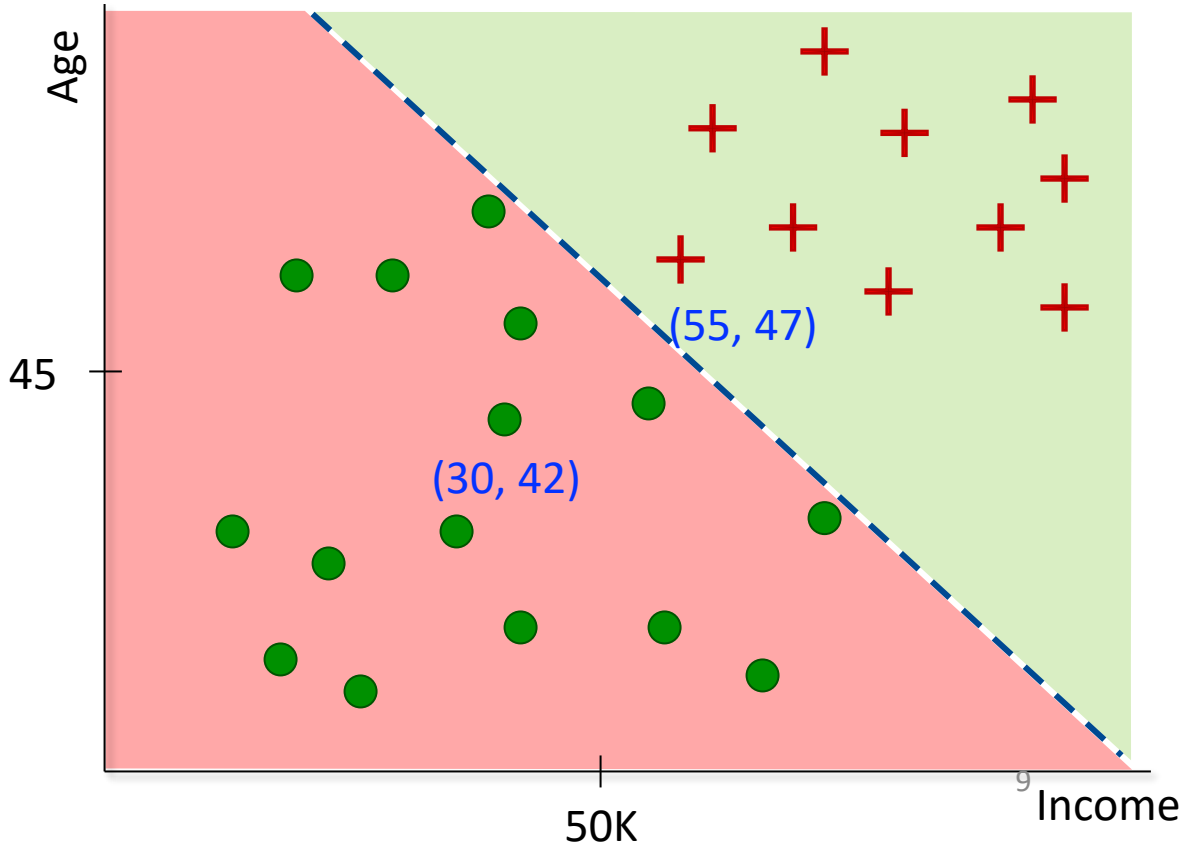
- Pass real value z to decision function for confidence of classification



Probability

- Many problems require a probability estimate as output $\hat{y}$
- Credit card application example
  - Probability of accepting application $p(accept|age, income)$
  - $z = age + 1.25 \times income - 80$
  - Probability $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$

age	Income (k)	z	$y = \sigma(z)$	class
47	55	35.75	0.9999	1
42	30	-0.5	0.3775	0



Interpretation of logistic regression

- $f(\boldsymbol{x})$ estimates the probability of class $p(y = 1|\boldsymbol{x}; \boldsymbol{w})$
- Example: cancer diagnosis from tumor size

$$\boldsymbol{x} = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{Tumor size} \end{bmatrix}$$

$$f(\boldsymbol{x}) = 0.7$$

The probability of this patient having malignant tumor is 70%

- Properties – probabilities of all classes sum up to 1

$$p(y = 1|\boldsymbol{x}; \boldsymbol{w}) + p(y = 0|\boldsymbol{x}; \boldsymbol{w}) = 1$$

Interpretation of logistic regression

- Learns the odds of positive class $y = 1$

$$\frac{p(y=1|\boldsymbol{x};\boldsymbol{w})}{p(y=0|\boldsymbol{x};\boldsymbol{w})}$$

- Take the log on odds

$$y = \frac{1}{1+e^{-z}}$$

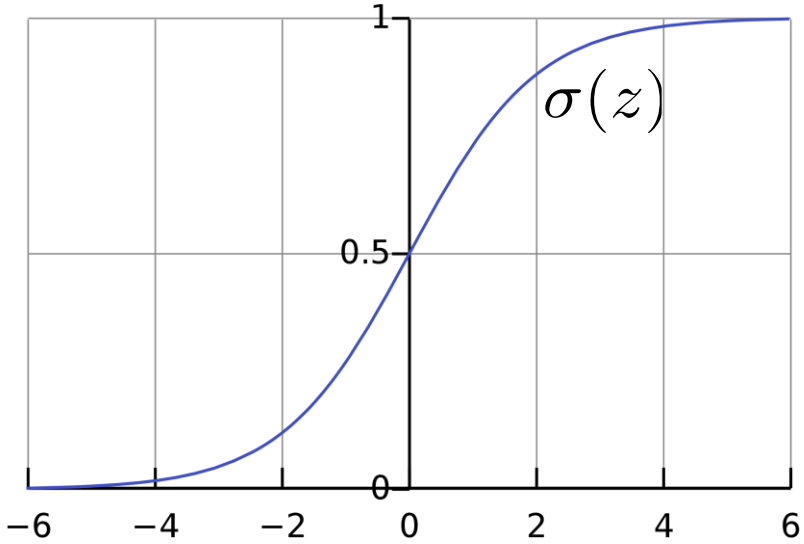
$$\log \frac{p(y=1|\boldsymbol{x};\boldsymbol{w})}{p(y=0|\boldsymbol{x};\boldsymbol{w})} = \log \frac{y}{1-y} = \log e^{z} = w_0 x_0 + w_1 x_1 + w_2 x_2 + \ldots + w_d x_d$$

- Logistic regression model assumes that the log odds is a linear function of $\boldsymbol{x}$

Decision boundary

$$f(\boldsymbol{x}) = \sigma(\boldsymbol{w}^T \boldsymbol{x})$$

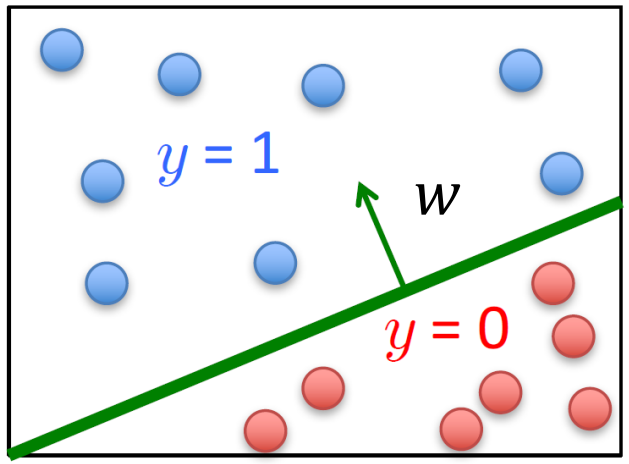
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



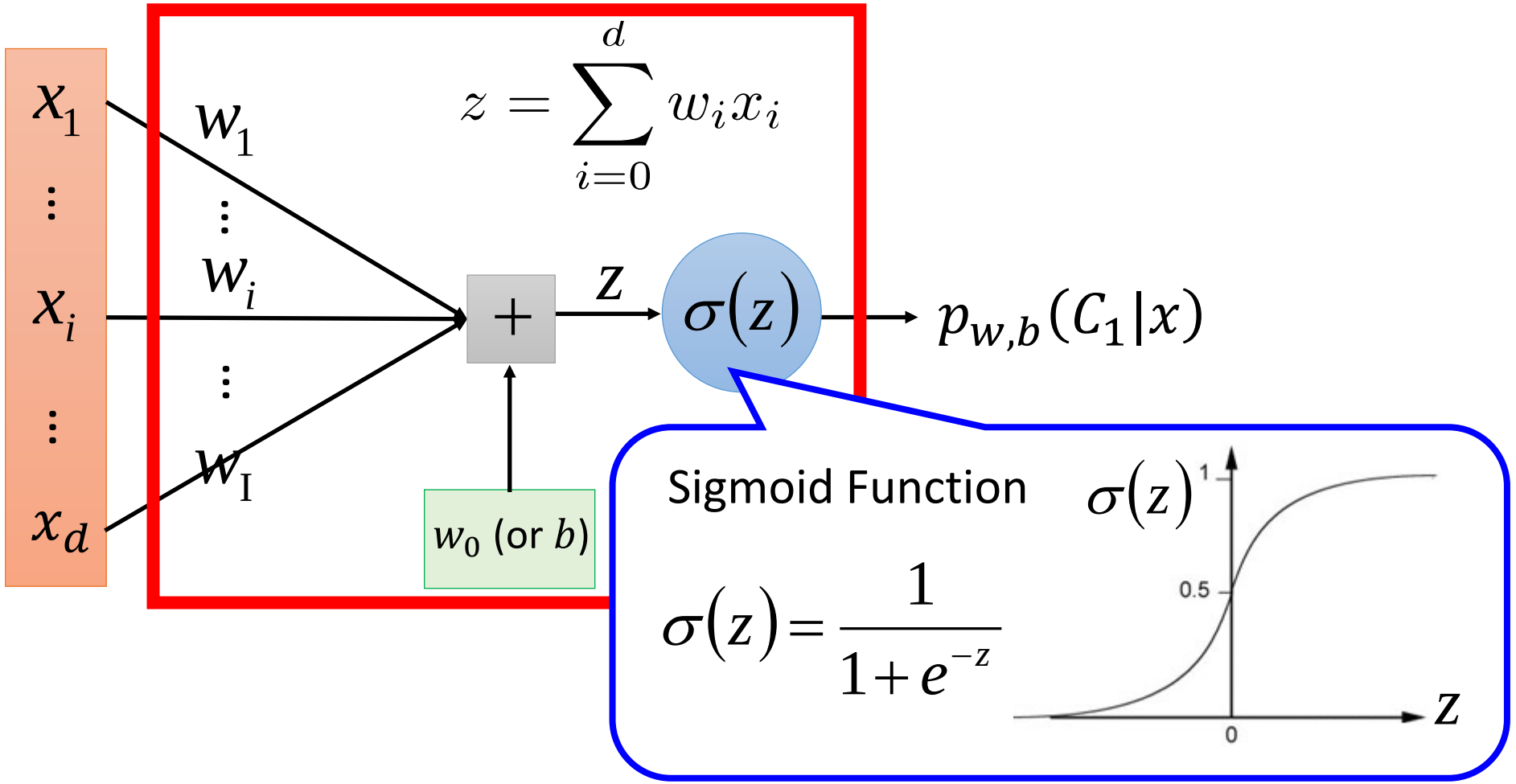
For class 0, $w^t x$ should be large negative values

For class 1, $w^t x$ should be large positive values

- Set a threshold that
  - Predict $y = 1$ if $f(x) \geq 0.5$
  - Predict $y = 0$ if $f(x) < 0.5$



Logistic regression as an artificial neuron: Connection to neural networks



Step 2: Goodness of a function
Learning logistic regression model

- How are the parameters of the model (the weights **w**) learned?
- We want to learn parameters **w** that make $\hat{y}$ for each training data as close as possible to the true class y

$$D = \{(\boldsymbol{x}^{(1)}, y^{(1)}), (\boldsymbol{x}^{(2)}, y^{(2)}), \ldots, (\boldsymbol{x}^{(n)}, y^{(n)})\}$$

$$\hat{y} = \sigma(\boldsymbol{w}^T \boldsymbol{x})$$

- Loss function: how close the classifier output ($\hat{y} = \sigma(\boldsymbol{w}^T \boldsymbol{x})$) is to the correct output (y, which is 0 or 1)

$\mathcal{L}(y, \hat{y})$ = How much predicted class $\hat{y}$ differs from the true y

Loss function in probability

- Maximum likelihood estimate give training data:

$$D = \{(\boldsymbol{x}^{(1)}, y^{(1)}), (\boldsymbol{x}^{(2)}, y^{(2)}), \dots, (\boldsymbol{x}^{(n)}, y^{(n)})\}$$

- Likelihood of a single data sample (given parameters)
  - How likely the features are to produce an observed sample?

$$p(y|\boldsymbol{x}) = \begin{cases} \dfrac{1}{1+e^{-(\boldsymbol{w}^T\boldsymbol{x})}}, & \text{if } y = 1 \\ 1 - \dfrac{1}{1+e^{-(\boldsymbol{w}^T\boldsymbol{x})}}, & \text{if } y = 0 \end{cases}$$

- Likelihood of the entire data is to multiply all sample $\prod_{i=1}^{n} p(y^{(i)}|\boldsymbol{x}^{(i)})$

Loss function in probability

- Maximum likelihood estimate give training data:

$$D = \{(\boldsymbol{x}^{(1)}, y^{(1)}), (\boldsymbol{x}^{(2)}, y^{(2)}), \dots, (\boldsymbol{x}^{(n)}, y^{(n)})\}$$

- Likelihood of a single data sample (given parameters)
  - How likely the features are to produce an observed sample?

$$p(y|\boldsymbol{x}) = \begin{cases} \dfrac{1}{1 + e^{-(\boldsymbol{w}^T \boldsymbol{x})}}, & \text{if } y = 1 \\ \dfrac{1}{1 + e^{(\boldsymbol{w}^T \boldsymbol{x})}}, & \text{if } y = 0 \end{cases}$$

- Likelihood of the entire data is to multiply all sample $\prod_{i=1}^{n} p(y^{(i)}|\boldsymbol{x}^{(i)})$

Deriving the loss function

- Likelihood of the data $D = \{(\boldsymbol{x}^{(1)}, y^{(1)}), (\boldsymbol{x}^{(2)}, y^{(2)}), \ldots, (\boldsymbol{x}^{(n)}, y^{(n)})\}$ given the parameters $\boldsymbol{w}$ is $\prod_{i=1}^{n} p(y^{(i)}|\boldsymbol{x}^{(i)}; \boldsymbol{w})$

$$\prod_{i=1}^{n} p(y^{(i)}|\boldsymbol{x}^{(i)}; \boldsymbol{w}) = \prod_{i=1}^{n} \left[p(y=1|\boldsymbol{x}^{(i)})^{y^{(i)}} \times p(y=0|\boldsymbol{x}^{(i)})^{1-y^{(i)}} \right]$$

Probability if **class 1**

Probability if **class 0**

$$p(y|\boldsymbol{x}) = \begin{cases} \dfrac{1}{1+e^{-(\boldsymbol{w}^T\boldsymbol{x})}}, & \text{if } y = 1 \\ \dfrac{1}{1+e^{(\boldsymbol{w}^T\boldsymbol{x})}}, & \text{if } y = 0 \end{cases}$$

Deriving the loss function

- Likelihood of the data $D = \{(\boldsymbol{x}^{(1)}, y^{(1)}), (\boldsymbol{x}^{(2)}, y^{(2)}), \ldots, (\boldsymbol{x}^{(n)}, y^{(n)})\}$ given the parameters $\boldsymbol{w}$ is $\prod_{i=1}^{n} p(y^{(i)}|\boldsymbol{x}^{(i)}; \boldsymbol{w})$

$$\prod_{i=1}^{n} p(y^{(i)}|\boldsymbol{x}^{(i)}; \boldsymbol{w}) = \prod_{i=1}^{n} \left[p(y=1|\boldsymbol{x}^{(i)})^{y^{(i)}} \times p(y=0|\boldsymbol{x}^{(i)})^{1-y^{(i)}} \right]$$

Probability if **class 1**

Probability if **class 0**

$$p(y|\boldsymbol{x}) = \begin{cases} \dfrac{1}{1+e^{-(\boldsymbol{w}^T\boldsymbol{x})}}, & \text{if } y = 1 \\ \dfrac{1}{1+e^{(\boldsymbol{w}^T\boldsymbol{x})}}, & \text{if } y = 0 \end{cases}$$

- Take log of both sides

$$\sum_{i=1}^{n} \log p(y^{(i)}|\boldsymbol{x}^{(i)}; \boldsymbol{w}) = \sum_{i=1}^{n} \left[y^{(i)} \log[p(y=1|\boldsymbol{x}^{(i)}) + (1-y^{(i)}) \log p(y=0|\boldsymbol{x}^{(i)}) \right]$$

Maximize the likelihood of the data, we get the best logistic regression model

Deriving the loss function

- Loss function (by adding a negative sign to likelihood)

$$L(\boldsymbol{w}) = -\sum_{i=1}^{n}\left[y^{(i)}\log p(y=1|\boldsymbol{x}^{(i)}) + (1-y^{(i)})\log p(y=0|\boldsymbol{x}^{(i)})\right]$$

$$= -\sum_{i=1}^{n}\left[y^{(i)}\log[\sigma(\boldsymbol{w}^T\boldsymbol{x}^{(i)})] + (1-y^{(i)})\log[1-\sigma(\boldsymbol{w}^T\boldsymbol{x}^{(i)})]\right]$$

Substitute probability p with discission function σ of $w^T x$

$$= -\sum_{i=1}^{n}\left[y^{(i)}(\boldsymbol{w}^T\boldsymbol{x}^{(i)}) - \log[1+e^{\boldsymbol{w}^T\boldsymbol{x}^{(i)}}]\right]$$

Substitute discission function σ with sigmoid function $1/(1+e^{w^T x})$

Finally, we show how loss function is determined by parameters w

Deriving the loss function

- Loss function (by adding a negative sign to likelihood)

$$L(\boldsymbol{w}) = -\sum_{i=1}^{n} \Big[y^{(i)} \log p(y=1|\boldsymbol{x}^{(i)}) + (1-y^{(i)}) \log p(y=0|\boldsymbol{x}^{(i)}) \Big]$$

$$= -\sum_{i=1}^{n} \Big[y^{(i)} \log[\sigma(\boldsymbol{w}^T \boldsymbol{x}^{(i)})] + (1-y^{(i)}) \log[1-\sigma(\boldsymbol{w}^T \boldsymbol{x}^{(i)})] \Big]$$

$$= -\sum_{i=1}^{n} \Big[y^{(i)} (\boldsymbol{w}^T \boldsymbol{x}^{(i)}) - \log[1+e^{\boldsymbol{w}^T \boldsymbol{x}^{(i)}}] \Big]$$

That's it, we got **cross-entropy loss** for classification!

Interpreting the loss function

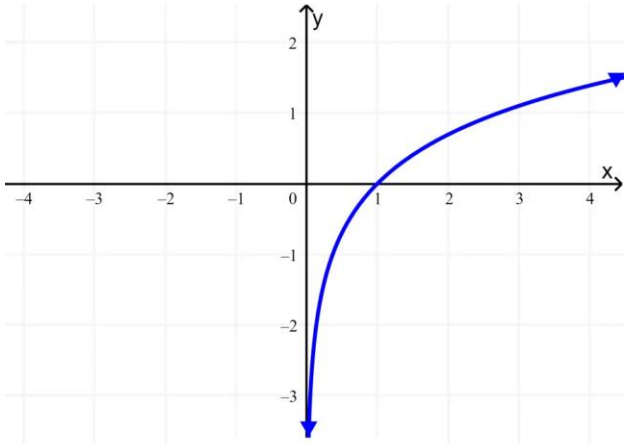
- Cross-entropy loss for classification

$$L(\boldsymbol{w}) = -\sum_{i=1}^{n} \left[y^{(i)} \log[f(\boldsymbol{x}^{(i)})] + (1 - y^{(i)}) \log[1 - f(\boldsymbol{x}^{(i)})] \right]$$

$\hat{y} = f(\boldsymbol{x})$ gives the prediction

- Loss of a single data instance i

$$L(\boldsymbol{w})^{(i)} = \begin{cases} -\log[f(\boldsymbol{x})], & \text{if } y = 1 \\ -\log[1 - f(\boldsymbol{x})], & \text{if } y = 0 \end{cases}$$

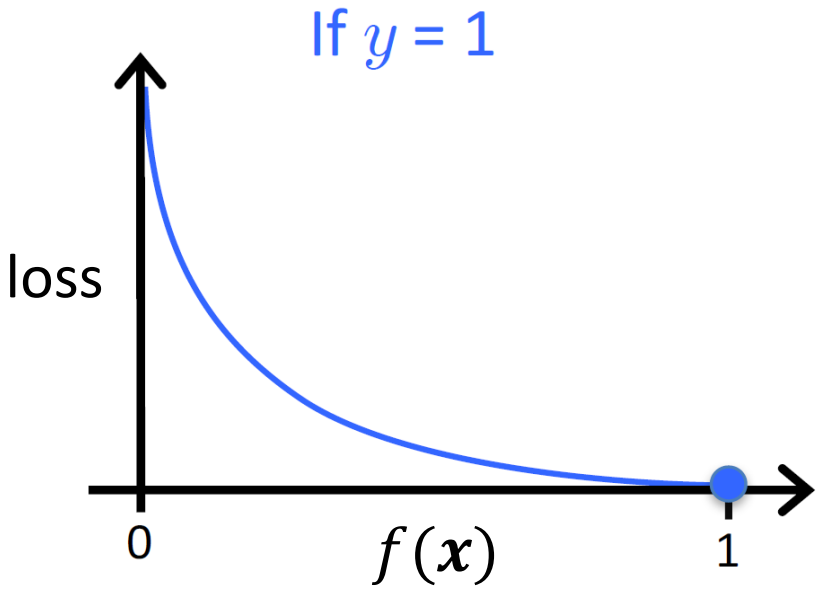


Plots of logarithm functions $\log_e(\cdot)$

Interpreting the Loss ..

- Loss of a single data instance

$$L(\boldsymbol{w})^{(i)} = \begin{cases} -\log[f(\boldsymbol{x})], & \text{if } y = 1 \\ -\log[1 - f(\boldsymbol{x})], & \text{if } y = 0 \end{cases}$$



When $y = 1$

- loss = 0 if prediction is correct
- Or $f(\boldsymbol{x}) \longrightarrow 0$, loss $\longrightarrow \infty$

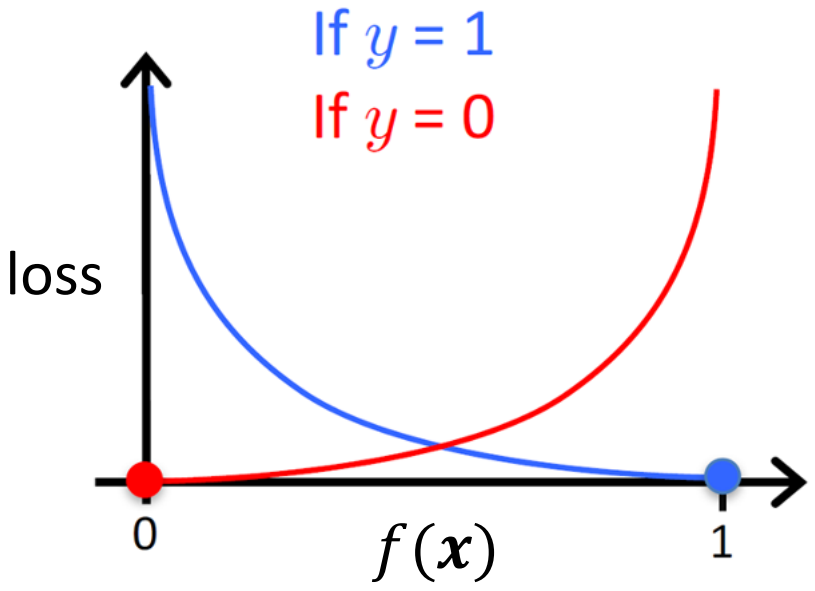
Intuition is that larger mistakes should get larger penalties

- e.g., predict $f(\boldsymbol{x}) = 0$, but $y = 1$

Interpreting the Loss ..

- Cost of a single data instance

$$L(\boldsymbol{w})^{(i)} = \begin{cases} -\log[f(\boldsymbol{x})], & \text{if } y = 1 \\ -\log[1 - f(\boldsymbol{x})], & \text{if } y = 0 \end{cases}$$



When $y = 0$

- loss = 0 if prediction is correct
- Or $(1 - f(\boldsymbol{x})) \longrightarrow 0$, loss $\longrightarrow \infty$

Larger mistakes get larger penalties as well

- e.g., predict $f(\boldsymbol{x}) = 1$, but $y = 0$

Regularized logistic regression

- Add a regularization term to constrain model complexity
  - Prevent overfitting as we do for linear regression
  - L2 norm ($||\cdot||_2$) - the square root of the sum of the squared vector values (Euclidean distance).
  - Measure the size of parameter vector $\boldsymbol{w}$

$$L_{\text{regularized}}(\boldsymbol{w}) = L(\boldsymbol{w}) + \lambda||\boldsymbol{w}||_2^2$$
$$= L(\boldsymbol{w}) + \lambda \sum_{j=1}^{d} w_j^2$$

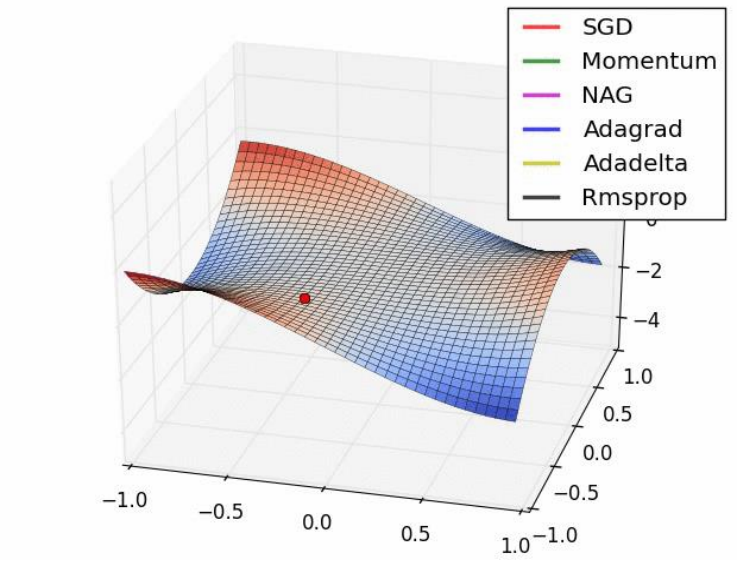
- Overall loss function for optimization

$$L_{\text{reg}}(\boldsymbol{w}) = -\sum_{i=1}^{n} \left[y^{(i)} \log[f(\boldsymbol{x}^{(i)})] + (1 - y^{(i)}) \log[1 - f(\boldsymbol{x}^{(i)})] \right] + \lambda \sum_{j=1}^{d} w_j^2$$

Step 3: Gradient descent for optimization

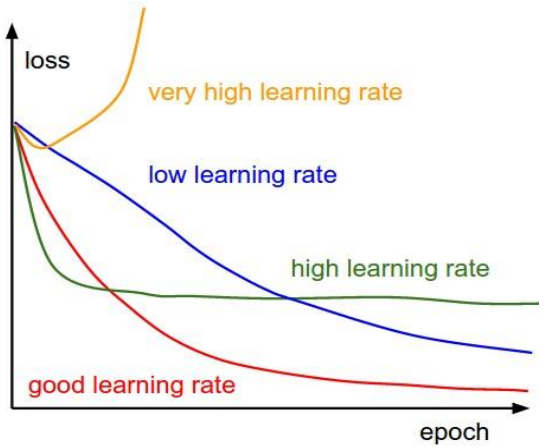
- To find the optimal weights: minimize the loss function we've defined for the model

$$\mathbf{w}^* = \underset{w}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} L\left(\boldsymbol{\sigma}(\mathbf{w}^\intercal \mathbf{x}^{(i)}), y^{(i)}\right)$$



- Learn by Gradient Descent method
  - Choose a starting point $[\boldsymbol{w}^0]$
  - Repeat until convergence
    - Compute gradient
    - Update parameters

$$w_j \leftarrow w_j - \alpha \frac{\partial}{\partial w_j} L(\boldsymbol{w}) \quad \text{update for } j = 0 \ldots d$$



Logistic Regression

Step 1: $f_{w,b}(x) = \sigma\left(\sum_i w_i x_i + b\right)$

Output: between 0 and 1

Step 2:

Step 3:

Linear Regression

$f_{w,b}(x) = \sum_i w_i x_i + b$

Output: real value

	Logistic Regression	***Linear Regression***
Step 1:	$f_{w,b}(x) = \sigma\left(\sum_i w_i x_i + b\right)$ Output: between 0 and 1	$f_{w,b}(x) = \sum_i w_i x_i + b$ Output: real value
Step 2:	Training data: (x, y) y: 1 for class 1, 0 for class 0 $L(f) = \sum_n L\left(f\left(x^{(n)}\right), y^{(n)}\right)$	Training data: (x, y) y: a real number $L(f) = \sum_n \left(f\left(x^{(n)}\right) - y^{(n)}\right)^2$ SSE loss

Cross entropy loss:

$$L\left(f\left(x^{(i)}\right), y^{(i)}\right) = -\left[y^{(i)} \log f\left(x^{(i)}\right) + \left(1 - y^{(i)}\right) \log\left(1 - f\left(x^{(i)}\right)\right)\right]$$

	Logistic Regression	***Linear Regression***
Step 1:	$f_{w,b}(x) = \sigma\left(\sum_i w_i x_i + b\right)$ Output: between 0 and 1	$f_{w,b}(x) = \sum_i w_i x_i + b$ Output: real value
Step 2:	Training data: (x, y) y: 1 for class 1, 0 for class 0 $L(f) = \sum_n L\left(f\left(x^{(n)}\right), y^{(n)}\right)$	Training data: (x, y) y: a real number $L(f) = \sum_n \left(f\left(x^{(n)}\right) - y^{(n)}\right)^2$
Step 3:	Logistic regression: $w_i \leftarrow w_i - \alpha \sum_n \left(f_{w,b}\left(x^{(n)}\right) - y^{(n)}\right) x_i^{(n)}$ Linear regression: $w_i \leftarrow w_i - \alpha \sum_n \left(f_{w,b}\left(x^{(n)}\right) - y^{(n)}\right) x_i^{(n)}$	

Demo

class sklearn.linear_model.SGDClassifier(*loss='log', *, penalty='l2', alpha=0.0001, l1_ratio=0.15, fit_intercept=True, max_iter=1000, tol=0.001, shuffle=True, verbose=0, epsilon=0.1, n_jobs=None, random_state=None, learning_rate='optimal', eta0=0.0, power_t=0.5, early_stopping=False, validation_fraction=0.1, n_iter_no_change=5, class_weight=None, warm_start=False, average=False*)

class sklearn.linear_model.LogisticRegression(*penalty='l2', *, dual=False, tol=0.0001, C=1.0, fit_intercept=True, intercept_scaling=1, class_weight=None, random_state=None, solver='lbfgs', max_iter=100, multi_class='auto', verbose=0, warm_start=False, n_jobs=None, l1_ratio=None*)

1. Prepare training and test data

```
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
```

2. Specific the model and train the model

```
clf = DecisionTreeClassifier(fit_intercept=True)
clf.fit(X_train, y_train)
```

3. Make prediction and evaluate the model

```
y_predict = clf.predict(X_test)
accuracy_score(y_test, y_predict)
```

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html#sklearn.linear_model.LogisticRegression

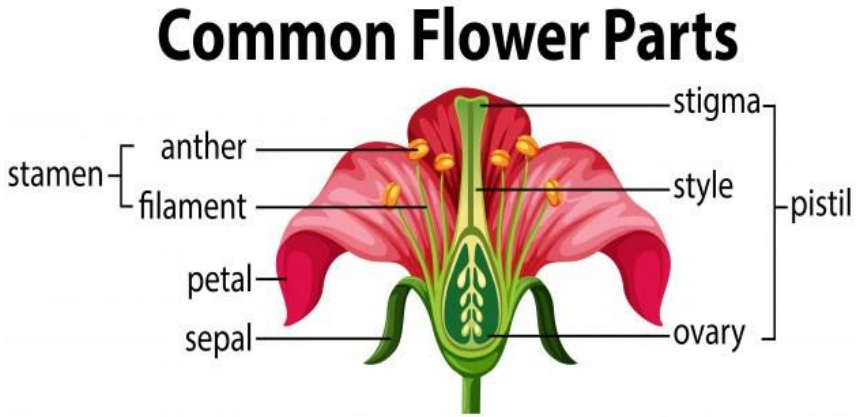
Iris sample data set

- Iris Plant data set.
  - Can be obtained from the UCI Machine Learning Repository http://www.ics.uci.edu/~mlearn/MLRepository.html
  - From the statistician Douglas Fisher
  - Three flower types (classes):
    - Setosa
    - Versicolour
    - Virginica
  - Four (non-class) attributes
    - Sepal width and length
    - Petal width and length

Setosa

Versicolour

Virginica



Data example

5.1,3.5,1.4,0.2,Iris-setosa
4.9,3.0,1.4,0.2,Iris-setosa
4.7,3.2,1.3,0.2,Iris-setosa
4.6,3.1,1.5,0.2,Iris-setosa
…

Agenda

- Logistic Regression model
  - Model definition
  - Loss function
  - Optimizing parameters
- Model Evaluation
  - Metrics
  - Methods

Model evaluation

- Metrics for Performance Evaluation
  - How to evaluate the predictive capability of a model?

- Methods for Evaluation Process
  - How to obtain reliable estimates?

Metrics for regression task

- How close is your prediction against the target value?

Mean square error (MSE)

$$MSE = \frac{1}{n}\sum_{i-1}^{n}(y^{(i)} - \hat{y}^{(i)})^2$$

Gives larger penalization to big prediction error by square it

Mean absolute error (MAE)

$$MAE = \frac{1}{n}\sum_{i-1}^{n}|y^{(i)} - \hat{y}^{(i)}|$$

Treats all error the same

- Can NOT interpret how the model performance from one single result
- But can be used to compare against other models

Metrics for regression task

- How close your prediction is against the target value

R-squared (R^2), also called coefficient of determination

$$R^2 = 1 - \frac{\sum_i (y^{(i)} - \hat{y}^{(i)})^2}{\sum_i (y^{(i)} - \bar{y}^{(i)})^2}$$

Prediction

Mean

A goodness-of-fit measure shows percentage of the target variation that a linear model explains

$$R^2 = \frac{\text{Variance explained by model}}{\text{Total variance}}$$

- More informative than MSE and MAE by showing percentage rather than absolute value (with arbitrary range)

Metrics for classification task

- Confusion Matrix (binary classification):

		PREDICTED CLASS	
		Class=Yes	Class=No
ACTUAL CLASS	Class=Yes	**a**	**b**
	Class=No	**c**	**d**

a: TP (true positive)

b: FN (false negative)

c: FP (false positive)

d: TN (true negative)

Metrics for performance evaluation

		PREDICTED CLASS	
		Class=Yes	Class=No
ACTUAL CLASS	Class=Yes	a (TP)	b (FN)
	Class=No	c (FP)	d (TN)

- Most widely-used metric:

$$\text{Accuracy} = \frac{a+d}{a+b+c+d} = \frac{TP+TN}{TP+TN+FP+FN}$$

Limitation of accuracy

- Consider a 2-class problem
  - Number of Class 0 examples = 9990
  - Number of Class 1 examples = 10

- If model predicts everything to be class 0, accuracy is 9990/10000 = 99.9 %
  - Accuracy is misleading because model does not detect any class 1 example

Cost matrix

		PREDICTED CLASS	
ACTUAL CLASS	C(i|j)	**Class=Yes**	**Class=No**
	Class=Yes	C(Yes|Yes)	C(No|Yes)
	Class=No	C(Yes|No)	C(No|No)

C(i|j): Cost of classifying class **j** example as class **i**

Weighted accuracy

CONFUSION MATRIX	PREDICTED CLASS		
ACTUAL CLASS		**Class=Yes**	**Class=No**
	Class=Yes	a (TP)	b (FN)
	Class=No	c (FP)	d (TN)

COST MATRIX	PREDICTED CLASS		
ACTUAL CLASS	C(i|j)	**Class=Yes**	**Class=No**
	Class=Yes	w_1 C(Yes|Yes)	w_2 C(No|Yes)
	Class=No	w_3 C(Yes|No)	w_4 C(No|No)

$$\text{Weighted Accuracy} = \frac{w_1 a + w_4 d}{w_1 a + w_2 b + w_3 c + w_4 d}$$

Computing weighted accuracy

Cost Matrix	PREDICTED CLASS		
ACTUAL CLASS	C(i|j)	+	-
	+	1	100
	-	1	1

Model M_1	PREDICTED CLASS		
ACTUAL CLASS		+	-
	+	150	40
	-	60	250

Accuracy = 80%

Weighted Accuracy = 8.9%

Model M_2	PREDICTED CLASS		
ACTUAL CLASS		+	-
	+	250	45
	-	5	200

Accuracy = 90%

Weighted Accuracy= 9%

Precision-Recall

Count	PREDICTED CLASS		
		Class=Yes	Class=No
ACTUAL CLASS	Class=Yes	a	b
	Class=No	c	d

$$\text{Precision (p)} = \frac{a}{a+c} = \frac{TP}{TP+FP}$$

$$\text{Recall (r)} = \frac{a}{a+b} = \frac{TP}{TP+FN}$$

$$\text{F-measure (F)} = \frac{1}{\left(\frac{1/r + 1/p}{2}\right)} = \frac{2rp}{r+p} = \frac{2a}{2a+b+c} = \frac{2TP}{2TP+FP+FN}$$

Assumption: The class YES is the one we care about.

Precision is biased towards **C(Yes|Yes) & C(Yes|No)**

Recall is biased towards **C(Yes|Yes) & C(No|Yes)**

F-measure is biased towards all except **C(No|No)**

ROC (Receiver Operating Characteristic)

- ROC curve plots the trade-off between TPR and FPR of a classifier
  - Changing threshold of the model to classify data (e.g., 0.5 for sigmoid function)

Look at the positive predictions of the classifier and compute:

$$TPR = \frac{TP}{TP + FN}$$

What fraction of positive instances are predicted correctly ?

$$FPR = \frac{FP}{FP + TN}$$

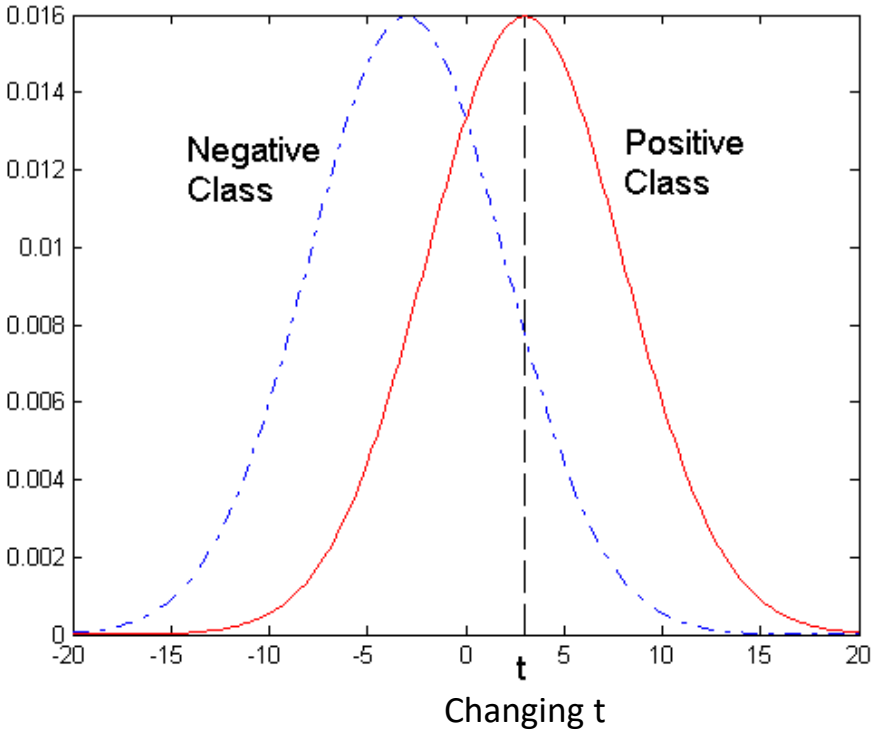
What fraction of negative instances were predicted incorrectly?

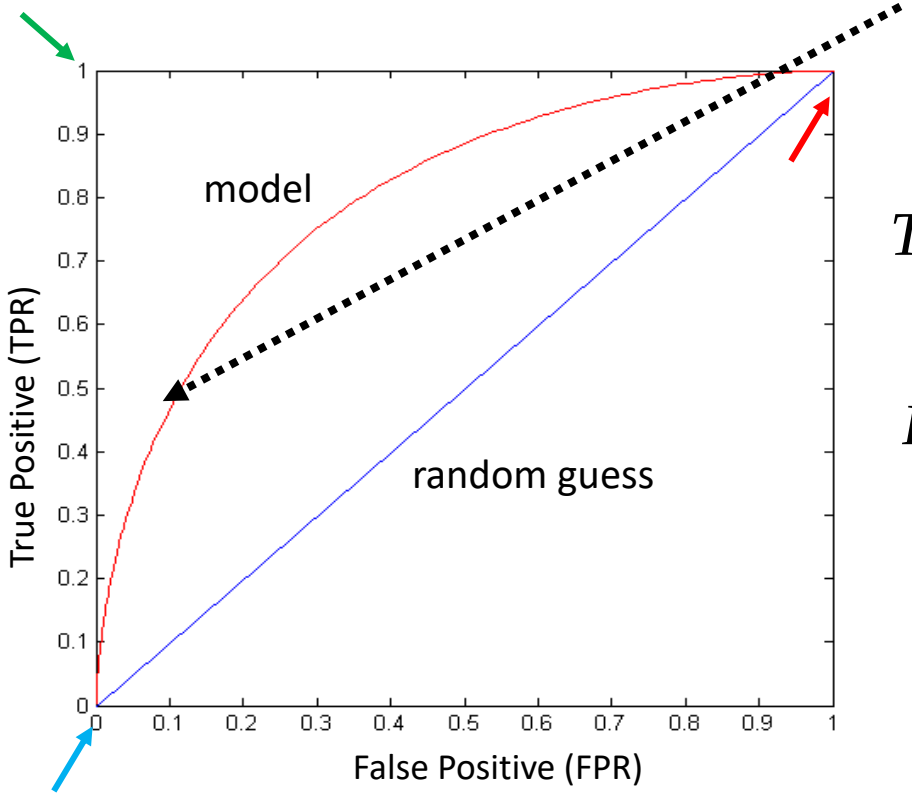
		Prediction	
		Yes	No
Actual	Yes	a (TP)	b (FN)
	No	c (FP)	d (TN)

ROC curve

- Data set containing **2** classes (***positive*** and ***negative***)

- Any points located at **x > t** is classified as ***positive***





At threshold **t**:

TPR=0.5, FPR=0.12

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

(**TPR=0, FPR=0**): Model predicts every instance to be a negative class

(**TPR=1, FPR=1**): Model predicts every instance to be a positive class

(**TPR=1, FPR=0**): The perfect model with zero misclassifications

Model evaluation

- Metrics for Performance Evaluation
  - How to evaluate the performance of a model?

- Methods for Evaluation Process
  - How to obtain reliable estimates?

Methods for evaluation process

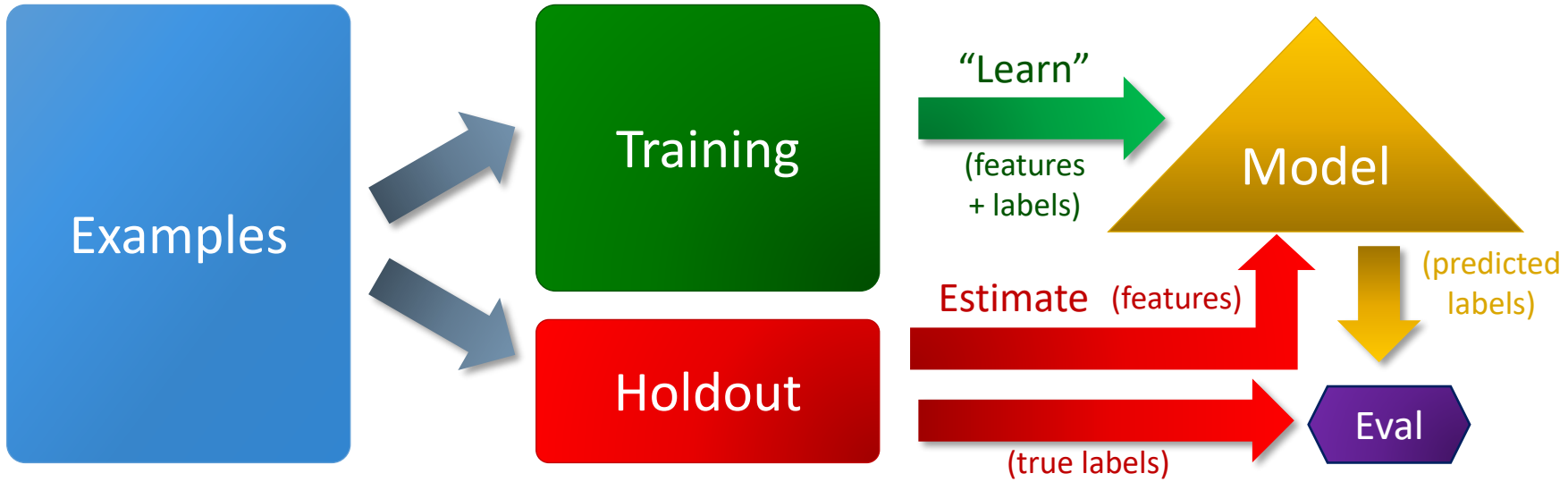
- How to obtain a reliable estimate of performance?

- Performance of a model may depend on other factors besides the learning algorithm:
  - Class distribution
  - Cost of misclassification
  - Size of training and test sets

Methods of estimation

- Holdout
  - Reserve two disjoint sets: training/testing (80%/20%)
  - One sample may be biased -- Repeated holdout by random subsampling
- Cross validation
  - Partition data into **k** disjoint subsets
  - **k**-fold: train on **k-1** partitions, test on the remaining one
  - Leave-one-out: **k=n**
  - Guarantees that each record is used the same number of times for training and testing
- Bootstrap
  - Sampling with replacement of size N
  - Repeat **b** times

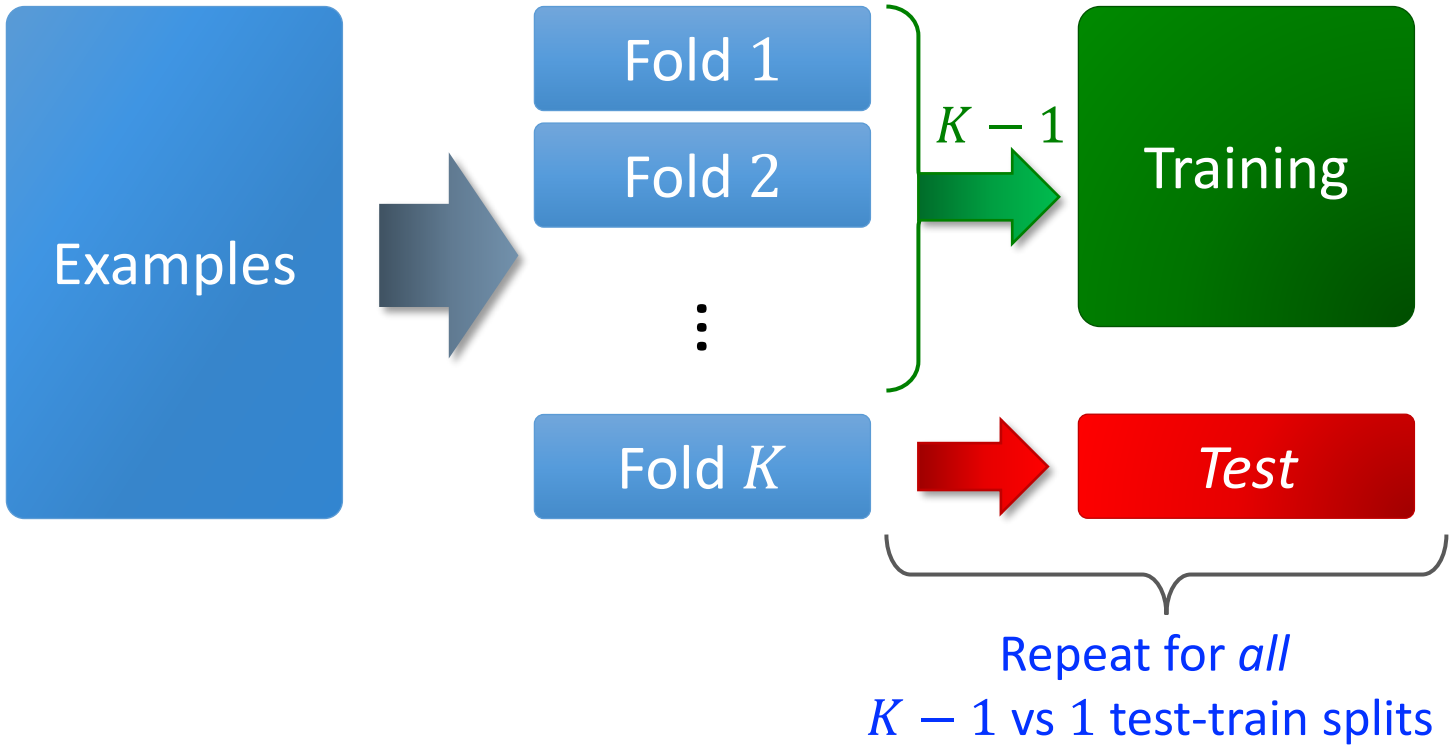
Holdout evaluation



- Train data on examples *training set* (not held out)
- Evaluate model on *holdout set* (aka test set)
- A small holdout set from training set is validation set, for monitoring overfitting

Cross-validation

- Why - data in test set is too different from data in training set

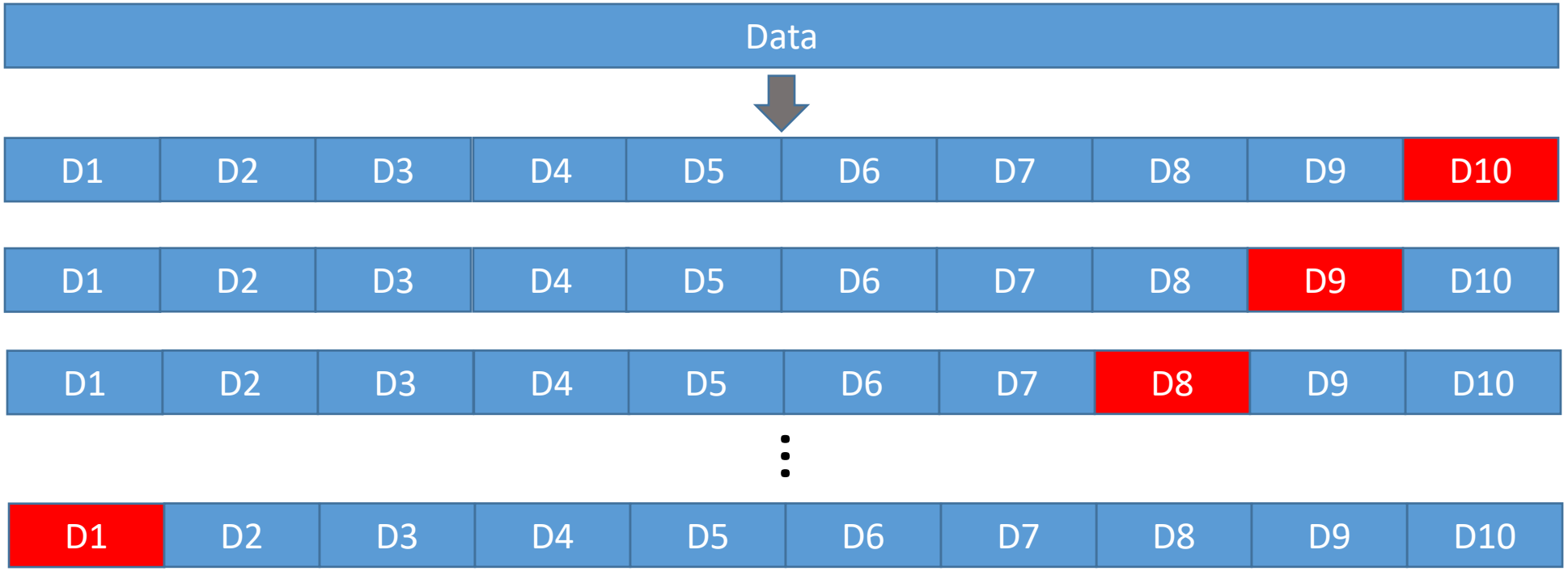


- Gives better estimate of generalization performance

Cross-validation

Common choices of K:

- 10-fold cross-validation: very common



- $(M - 1)$-fold cross-validation (where M is #examples):
  - Aka "leave one out" cross-validation

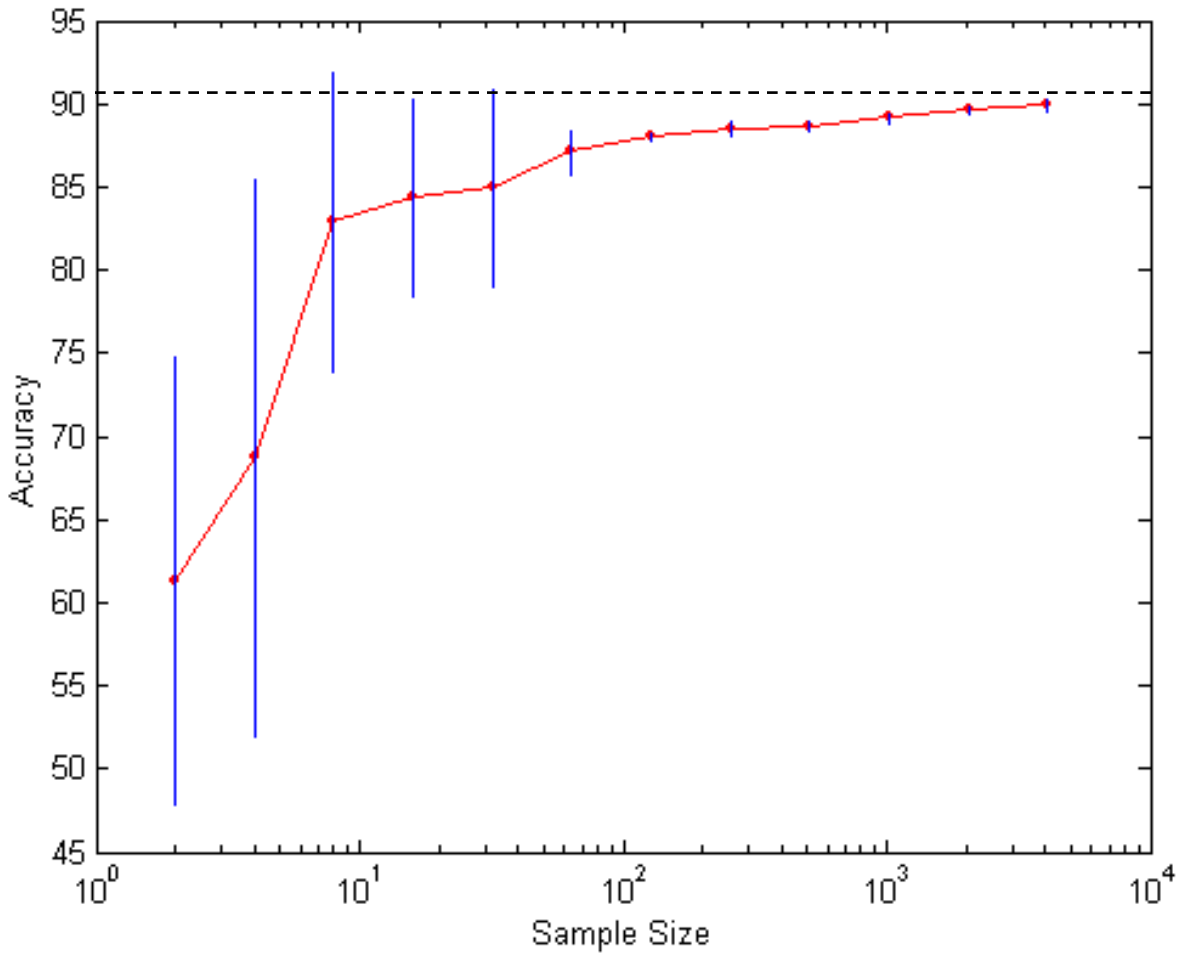
Variations on cross-validation

- Repeated cross-validation
  - Perform cross-validation a number of times
  - Gives an estimate of the variance of the generalization error
- Stratified cross-validation
  - Guarantee the same percentage of class labels in training and test
  - Important when classes are imbalanced and the sample is small

Dealing with class imbalance

- If the class we are interested in is very rare, then the classifier will ignore it.
  - The class imbalance problem
- Solution
  - We can modify the optimization criterion by using a cost sensitive metric
  - We can balance the class distribution
    - Sample from the larger class so that the size of the two classes is the same
    - Replicate the data of the class of interest so that the classes are balanced

Learning curve



Learning curve shows how accuracy changes with varying sample size

Requires a sampling schedule for creating learning curve

Effect of small sample size:

- Bias in the estimate
  - Poor model
- Variance of estimate
  - Poor training data